

CO2-AT1- Analytical Problem Solving

Q2. Local vs. Global Histogram Stretching

TOOLS USED: Python, OpenCV

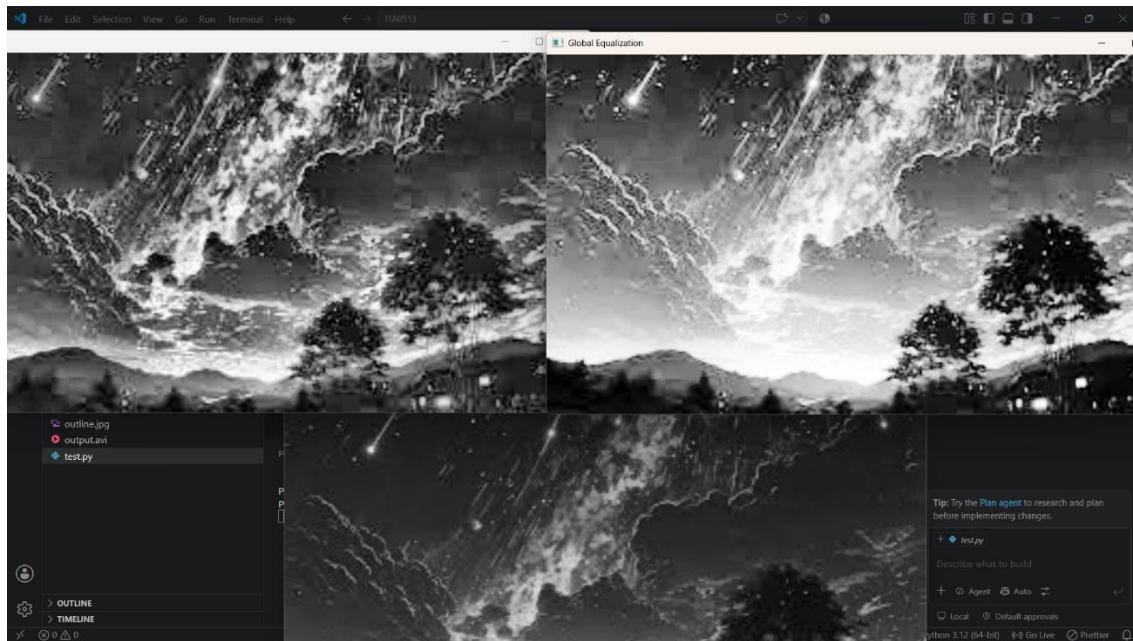
INPUT IMAGE:



CODE:

```
1 import cv2
2
3 # Read image
4 img = cv2.imread("ex8.jpg", 0)
5
6 # Global Histogram Equalization
7 global_eq = cv2.equalizeHist(img)
8
9 # CLAHE
10 clahe = cv2.createCLAHE(clipLimit=4.0, tileGridSize=(8,8))
11 clahe_img = clahe.apply(img)
12
13 cv2.imshow("Original", img)
14 cv2.imshow("Global Equalization", global_eq)
15 cv2.imshow("CLAHE", clahe_img)
16
17 cv2.waitKey(0)
18 cv2.destroyAllWindows()
```

OUTPUT IMAGE:



Description:

The output demonstrates that Global Histogram Equalization improves the overall image contrast but may over-enhance bright regions. In contrast, Local Histogram Equalization (CLAHE) provides better local contrast enhancement, preserving fine details and producing a more visually balanced image, especially in regions with varying illumination.

Q4. Unsharp Masking & High-Boost Tuning

TOOLS USED: Pixlane - Unsharp Mask Tool

INPUT IMAGE:



OUTPUT IMAGE:



Description:

The unsharp masking process successfully improves image sharpness by enhancing edge contrast while preserving the overall appearance. Fine details become clearer, although minor white and black halo artifacts may be visible near strong edges.

6. 2D Fast Fourier Transform (FFT) Phase Filtering

TOOLS USED: Python, OpenCV, Numpy, Matplotlib

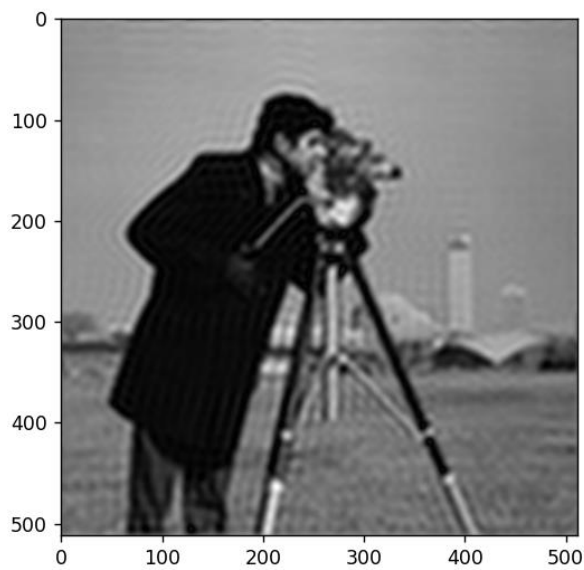
INPUT IMAGE:



CODE:

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 img = cv2.imread("Q6.png",0)
6
7 f = np.fft.fft2(img)
8 fshift = np.fft.fftshift(f)
9
10 rows, cols = img.shape
11 mask = np.zeros((rows,cols),np.uint8)
12
13 r = 40
14 cx, cy = cols//2, rows//2
15 mask[cy-r:cy+r,cx-r:cx+r] = 1
16
17 filtered = fshift * mask
18
19 img_back = np.fft.ifft2(np.fft.ifftshift(filtered))
20 img_back = np.abs(img_back)
21
22 plt.imshow(img_back,cmap='gray')
23 plt.show()
```

OUTPUT IMAGE:



Description:

The 2D Fast Fourier Transform (FFT) converts an image from the spatial domain to the frequency domain. By removing the high-frequency components located far from the center of the FFT spectrum, fine details and edges are suppressed. The reconstructed image becomes smoother and blurred, producing an effect similar to a Gaussian low-pass filter while preserving the overall structure of the image.

Q8. Python OpenCV Edge Enhancement Sandbox

TOOLS USED: Python, OpenCV, Numpy, Matplotlib

INPUT IMAGE:



CODE:


```

1  import cv2
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # Read image
6  img = cv2.imread("bricks.jpg", cv2.IMREAD_GRAYSCALE)
7
8  # Sobel 3x3
9  sobelx3 = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
10 sobely3 = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
11
12 gradient3 = np.sqrt(sobelx3**2 + sobely3**2)
13
14 laplacian = cv2.Laplacian(img, cv2.CV_64F)
15
16 # Sobel 5x5
17 sobelx5 = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
18 sobely5 = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
19
20 gradient5 = np.sqrt(sobelx5**2 + sobely5**2)
21
22 plt.figure(figsize=(15,10))
23
24 plt.subplot(2,4,1)
25 plt.imshow(img,cmap='gray')
26 plt.title("Original")
27 plt.axis("off")
28
29 plt.subplot(2,4,2)
30 plt.imshow(sobelx3,cmap='gray')
31 plt.title("Sobel X (3x3)")
32 plt.axis("off")
33
34 plt.subplot(2,4,3)
35 plt.imshow(sobely3,cmap='gray')
36 plt.title("Sobel Y (3x3)")
37 plt.axis("off")
38

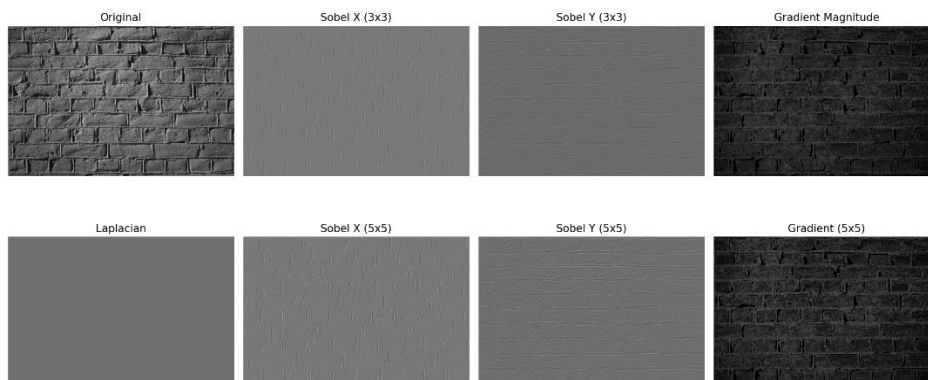
```

```

39 plt.subplot(2,4,4)
40 plt.imshow(gradient3,cmap='gray')
41 plt.title("Gradient Magnitude")
42 plt.axis("off")
43
44 plt.subplot(2,4,5)
45 plt.imshow(laplacian,cmap='gray')
46 plt.title("Laplacian")
47 plt.axis("off")
48
49 plt.subplot(2,4,6)
50 plt.imshow(sobelx5,cmap='gray')
51 plt.title("Sobel X (5x5)")
52 plt.axis("off")
53
54 plt.subplot(2,4,7)
55 plt.imshow(sobely5,cmap='gray')
56 plt.title("Sobel Y (5x5)")
57 plt.axis("off")
58
59 plt.subplot(2,4,8)
60 plt.imshow(gradient5,cmap='gray')
61 plt.title("Gradient (5x5)")
62 plt.axis("off")
63
64 plt.tight_layout()
65 plt.show()

```

OUTPUT IMAGE:



Description:

In this experiment, OpenCV is used to detect image edges using the **Sobel X**, **Sobel Y**, and **Laplacian** operators. The gradient magnitude is computed to highlight edges in all directions. Increasing the Sobel kernel size from **3×3** to **5×5** produces thicker and smoother edges while reducing noise, but some fine details become less sharp.